

第10章: デプロイと総まとめ

おめでとうございます！最終章です。この章では、ローカル（自分のPC）で動いているアプリをインターネット上に公開して、誰でもアクセスできるようにします。

この章で行うこと

作業	内容	所要時間の目安
ビルド	Build：開発用コードを本番環境用に最適化する。ファイルサイズの圧縮やコードの最適化が行われる	5分
デプロイ	Deploy：最適化したコードをサーバーに配置して公開する。「あなたのアプリにURLが付いて、世界中からアクセスできる状態になる」こと	10～15分
本番設定	Supabaseのセキュリティ設定の確認と調整	10分
振り返り	この教材で学んだことの総まとめ	10分

デプロイとは？「デプロイ（Deploy）」は英語で「配置する／配備する」という意味の単語です。プログラミングの世界では、**自分のPCで動いているアプリを、インターネット上のサーバーに置いて、URLを叩けば世界中の誰でも使える状態にする作業**を指します。自分のPCで動いているアプリは、他の人からはアクセスできません。デプロイをすると、**https://～** のURLが発行され、スマホからでも、海外からでもアクセスできるようになります。レストランで例えると、自宅で試作した料理を実店舗で提供開始する段階です。

この章では、**Vercel**（ヴァーセル：Next.jsを開発している会社が提供するホスティングサービス。GitHubと連携するだけで自動デプロイできる）を使ってデプロイします。そして、この教材全体を通じて学んだことを振り返り、今後の学習ロードマップ（次に何を学ぶべきかの道筋）を示します。

ホスティング（Hosting）とは？「Host（ホスト）」は「招き入れる人」という意味です。自分が作ったWebアプリやファイルを、24時間動いているサーバー（コンピューター）に置いて、世界中からアクセスできるようにするサービスを「ホスティングサービス」と呼びます。Vercel、Netlify、AWS などが代表例です。自分のPCはずっと電源を入れっぱなしにできないので、専門の会社のサーバーを借りる、というイメージです。

サーバレス（Serverless）とは？直訳すると「サーバーがない」ですが、「サーバーがない」のではなく「サーバーの存在を意識なくていい」という意味です。従来は自分でサーバーを用意・管理する必要がありましたが、サーバレスでは「コードを置くだけ」で、必要なときに自動でサーバーが動き、終わったら止まります。料金もアクセスがあった分だけ。Vercel はこのサーバレスの仕組みでアプリを動かしています。

Edge Function（エッジ関数）とは？「Edge」は「端っこ」という意味で、ここでは世界中に散らばっているサーバー（ユーザーに近い場所）を指します。Edge Function は、そのユーザーに一番近いサーバーで実行されるプログラムのことで、応答が速いのが特徴です。Vercel ではこの仕組みでページを高速配信しています。

目次

- 0. 前提知識: ビルド／デプロイ／環境変数とは
 - 1. デプロイの準備
 - 2. Vercel へのデプロイ
 - 3. Supabase の本番設定
 - 4. パフォーマンス最適化
 - 5. 学習の振り返り
 - 6. 次のステップ（発展学習）
 - 7. 推奨学習リソース
 - 8. 付録: 全ファイルの完成版一覧
 - 9. おわりに

0. 前提知識: ビルド／デプロイ／環境変数とは

0.1 ビルド（Build）とは

「書いたコードを、本番で実行できる形に変換する」処理です。 `npm run build` で実行します。具体的には:

- 1. TypeScript → JavaScript に変換（ブラウザは TS を読めない）
- 2. JSX → 通常のJS関数呼び出しに変換
- 3. ファイルを圧縮（不要な空白・コメント削除、変数名を短縮）
- 4. ファイルを結合（バンドル）してリクエスト数を削減

5. 画像やフォントを最適化

▼ ビルド実行例:

```
# $ はターミナルの入力待ち記号 (自分でタイプする必要はない)
# npm = Node.js に付属するパッケージマネージャ
# run = package.json の scripts に書かれたコマンドを実行する命令
# build = package.json で定義された「ビルド用」のスクリプト名 (中身は "next build")
$ npm run build
```

▼ 期待される出力 (抜粋):

```
> next build

  ▲ Next.js 15.0.0
  - Environments: .env.local

  Creating an optimized production build ...
✓ Compiled successfully in 12.3s
  Linting and checking validity of types
✓ Generating static pages (5/5)
  Finalizing page optimization

Route (app)                Size      First Load JS
┌ ○ /                      1.32 kB    92.5 kB
├ ○ /books                 2.45 kB    93.6 kB
├ λ /books/[id]           1.78 kB    92.9 kB
└ λ /books/[id]/edit      2.10 kB    93.2 kB

○ (Static)   prerendered as static content
λ (Dynamic)  server-rendered on demand
```

▼ 何が起きた? - すべてのページが正常にビルドされた - 各ページのファイルサイズが表示される - ○ は静的ページ (あらかじめHTMLが用意されている)、λ はリクエストごとにサーバーで生成されるページ - **First Load JS** は「最初にユーザーのブラウザがダウンロードする JavaScript の量」。小さいほど読み込みが速い

ビルド失敗が出たら? TypeScriptの型エラーや lint エラーが残っているとビルドできません。エラーメッセージのファイル名と行番号を見て修正します。**ローカルでビルドが通ってからデプロイする習慣**をつけると安全です。本番のVercel上でビルドに失敗すると、せっかく **git push** しても新しいバージョンが反映されないため、必ず手元で **npm run build** を1回通してからプッシュしましょう。

0.2 デプロイ（Deploy）とは

ビルドの成果物をインターネット上のサーバーに配置して、誰でも URL でアクセスできるようにする作業です。本書では Vercel に GitHub 経由で自動デプロイします。



CI/CD（シーアイ・シーディー）とは？「Continuous Integration（継続的インテグレーション）」と「Continuous Delivery / Deployment（継続的デリバリー／デプロイ）」の略。コードを変更するたびに、テスト・ビルド・デプロイを自動で行う仕組みのことです。Vercel + GitHub の組み合わせも、簡単な CI/CD の一種で、`git push` するだけで自動的にビルドからデプロイまで進みます。手作業のミスを減らせるのが大きなメリットです。

プレビュー環境（Preview Environment）とは？ 本番に反映する前に、変更内容を試せる「お試し版」の URL のこと。Vercel では、Pull Request（変更案の提案）を作るたびに自動で `https://your-app-git-feature-xxx.vercel.app` のような専用 URL が生成され、本番のデータには影響を与えずに動作確認ができます。

ロールバック（Rollback）とは？「Roll（巻く）」+「Back（戻す）」で、「変更を取り消して前の状態に戻す」こと。例えば、新しくデプロイしたバージョンに不具合があった場合、Vercel のダッシュボードからボタン1つで以前動いていたバージョンに戻せます。これがあるので、安心して新しいバージョンを試せます。

ドメイン／DNS／SSLとは？ - **ドメイン（Domain）**：`google.com` や `your-app.vercel.app` のような、人間が覚えやすいインターネット上の住所のこと。本来サーバーは `192.168.x.x` のような数字（IPアドレス）で識別されますが、覚えるのが大変なので、文字列の住所を割り当てます。 - **DNS（Domain Name System）**：ドメイン名と IPアドレスを変換してくれる「インターネットの電話帳」。「`your-app.vercel.app` ってどこ？」と聞くと、「IPアドレスは〇〇です」と教えてくれます。 - **SSL/TLS**：通信を暗号化する技術。「Secure Sockets Layer」「Transport Layer Security」の略。URL が `http://` ではなく `https://` で始まっていれば SSL/TLS で暗号化されている証拠で、鍵マークがブラウザに表示されます。Vercel では自動的に有効になっています。

0.3 環境変数（Environment Variables）とは

「コードに直接書きたくない設定値」を別の場所で管理する仕組みです。代表例：

変数名（例）	用途	機密性
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase のURL	公開可（ブラウザに送られる）
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase の匿名キー	公開可
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Supabaseの管理者キー	絶対秘密（サーバー専用）

ローカル: `.env.local` ファイルに書く（`.gitignore` でGit管理から外れている）

```
# .env.local（このファイルはGitHubに上げない！）
# 「#」で始まる行はコメント。実行時には無視される
# 「変数名=値」の形式で1行に1つの環境変数を書く（=の左右にスペースを入れない）

NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
# ↑ NEXT_PUBLIC_ プレフィックス付きなのでブラウザにも値が送られる。URL は公開しても問題ない情報

NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ...
# ↑ 匿名キー。これも公開前提のキー。ただし RLS（Row Level Security）でデータを保護することが必須
```

本番（Vercel）: Vercelダッシュボードの **Settings** → **Environment Variables** で同じ値を設定する。

NEXT_PUBLIC_ の意味: 接頭辞 `NEXT_PUBLIC_` を付けた環境変数は**ブラウザ側にも送られる**（＝世界に公開される）。付けない変数はサーバーでしか参照できないので、シークレットには `NEXT_PUBLIC_` を付けてはいけません。Next.js は、ビルド時に `NEXT_PUBLIC_` 付きの環境変数を JavaScript ファイルの中に埋め込みます。そのためビルド後にブラウザの開発者ツールでソースを開くと、その値が文字列として見えてしまいます。逆に `NEXT_PUBLIC_` が付いていない変数は、サーバー上でしか参照できず、ブラウザには出力されません。

環境変数を変更したら必ず「再デプロイ」が必要な理由: Next.js はビルド時に環境変数の値をコードの中に焼き付けます。そのため、Vercel ダッシュボードで環境変数の値を変えただけでは反映されません。**変更後は「Redeploy」ボタンを押して、新しい値で再ビルドする必要がある**ということを覚えておきましょう。これは初心者が「変えたのに反映されない！」とハマる典型ポイントです。

0.4 開発・本番の3つの違い

項目	開発 (npm run dev)	本番 (npm run build → vercel)
URL	<code>http://localhost:3000</code>	<code>https://your-app.vercel.app</code>
速度	遅い (毎回コンパイル)	速い (最適化済み)
エラー表示	詳細なスタックトレース	ユーザーには非表示
ホットリロード	あり (保存で即反映)	なし
ファイルサイズ	大きい	小さい (圧縮済み)
環境変数の出元	<code>.env.local</code>	Vercel の設定画面

ローカルと本番でハマるポイント: - 環境変数の設定漏れ: `.env.local` には書いたが、Vercel 側に登録し忘れる。本番で「supabaseUrl is required」エラーになる典型例 - 大文字小文字の違い: Windows/Mac はファイル名の大文字小文字を区別しないが、Vercel (Linux) は厳密に区別する。`BookCard.tsx` と `bookcard.tsx` は別ファイル扱い - `window` や `localStorage` をサーバー側で参照: 開発時には気づかなくても、本番のServer Componentでは `window is not defined` エラーになる - タイムゾーン: ローカル (日本時間) と Vercel のサーバー (UTC) で日時の扱いが違う場合がある - ビルド時にしか実行されないコード: `generateStaticParams` などはローカルでは毎回実行されるが、本番では1回しか実行されないので注意

1. デプロイの準備

アプリケーションを本番環境にデプロイする前に、いくつかの準備が必要です。ローカル環境では動いていても、本番環境では動かないケースがあるため、事前にしっかり確認しましょう。

1.1 デプロイまでの全体フロー

まず、開発からデプロイまでの全体的な流れを確認しましょう。



1.2 ビルドの実行

Next.js アプリケーションをデプロイする前に、必ずローカルでビルドを実行して問題がないか確認します。

```
# プロジェクトのルートディレクトリ (package.json があるフォルダ) で実行する
# npm = Node Package Manager (Node.js のパッケージ管理ツール)
# run = scripts に書かれたコマンドを実行する
# build = scripts の "build" を実行する (実体は "next build")
npm run build
```

ビルドが成功すると、以下のような出力が表示されます。

```
Route (app)                                Size      First Load JS
├─ /                                         5.2 kB      89.4 kB
├─ /books                                   3.1 kB      87.3 kB
├─ /books/[id]                             2.8 kB      87.0 kB
├─ /books/new                              4.5 kB      88.7 kB
├─ /books/[id]/edit                        4.7 kB      88.9 kB
+ First Load JS shared by all              84.2 kB
  ├ chunks/framework-XXXXX.js              45.2 kB
  ├ chunks/main-XXXXX.js                   31.8 kB
  └ other shared chunks (total)             7.2 kB

○ (Static) prerendered as static content
```

ビルドとは？ 開発中のコード（TypeScript、JSX など）をブラウザが直接理解できる形式（JavaScript、HTML、CSS）に変換し、最適化する処理のことです。ビルドによってファイルサイズが小さくなり、読み込み速度が向上します。

1.3 よくあるビルドエラーと解決方法

ビルド時にエラーが出ることはよくあります。エラーが出たからといって自分のスキルを疑う必要はなく、エラーは「どこを直せばよいか教えてくれるヒント」だと考えましょう。以下は、初心者がよく遭遇するエラーとその解決方法です。

ビルドエラーメッセージの読み方

エラーメッセージは英語で出ますが、構造はだいたい同じです。次の3点を順番に確認すれば、ほとんどのエラーは特定できます。

1. エラーの種類: `Type error`、`Module not found`、`SyntaxError` など、一番上の行
2. ファイル名と行番号: `./src/components/BookCard.tsx:12:5` のように出る部分（12行目の5文字目）
3. エラーの詳細メッセージ: 何が問題なのか具体的に書かれている部分

エラー1: TypeScript の型エラー

```
Type error: Property 'title' does not exist on type 'Book'.
```

原因: 型定義と実際のコードが一致していない。

解決方法:

```
// 型定義を確認して修正する
// type は「型エイリアス」と呼ばれ、Book という名前で型を定義する構文
type Book = {
  // 書籍ID。文字列型 (UUID形式の文字列を想定)
  id: string;
  // ← このプロパティが定義されているか確認。書籍タイトル
  title: string;
  // 著者名。文字列型
  author: string;
  // 評価。数値型 (1~5を想定)
  rating: number;
  // 作成日時。Supabase が ISO 8601 形式の文字列で返す
  created_at: string;
};
```

エラー2: import エラー

```
Module not found: Can't resolve '@components/BookCard'
```

原因: ファイルパスが間違っている、またはファイルが存在しない。

解決方法:

```
# ls = list (一覧表示) コマンド。指定したファイルが存在するか確認するために使う
# 存在しない場合は「No such file or directory」と表示される
ls src/components/BookCard.tsx

# ファイル名の太文字・小文字も確認 (Linux ではケースセンシティブ)
# BookCard.tsx と bookCard.tsx は別のファイルとして扱われます
# Windows/Mac の開発環境では区別されないため、ローカルで動いても Vercel (Linux) で失敗する典型例
```

エラー3: 環境変数が undefined

```
Error: supabaseUrl is required.
```

原因: 環境変数が設定されていない、または `.env.local` が読み込まれていない。

解決方法:

```
# cat = ファイルの中身を表示するコマンド (concatenate の略)
# .env.local の中身を一通りで確認できる
cat .env.local

# 以下の変数が設定されているか確認
# NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
# NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIs...
# 注意: 環境変数を書き換えた後は、開発サーバー (npm run dev) を停止して再起動が必要
```

エラー4: 'use client' ディレクティブの不足

```
Error: useState only works in Client Components. Add the "use client" directive.
```

原因: Client Component のフック (`useState`, `useEffect` など) を Server Component で使っている。

解決方法:

```
// ファイルの先頭 (必ず1行目) に追加
// この一行があると、Next.js はこのファイルを Client Component として扱う
// Server Component (デフォルト) はサーバー側でしか動かないので、useState や useEffect が使えない
'use client';

// React の useState フックを使うために import する
import { useState } from 'react';
// ...
```

エラー5: ESLint エラー

```
ESLint: 'variable' is defined but never used. (@typescript-eslint/no-unused-vars)
```

原因: 使用されていない変数やインポートがある。

解決方法:

```
// 不要な変数・インポートを削除する
// または、意図的に未使用の場合はアンダースコアを付けると ESLint が無視してくれる
// アンダースコア ( _) 始まりの変数は「使わないことを明示している」という慣習
const _unusedVariable = 'something';
```

その他、本番でハマる典型エラー: - `Hydration failed because the initial UI does not match...`: サーバー側で生成したHTMLとクライアント側のレンダリング結果が違うときに出る。日時表示やランダム値が原因のことが多い - `Cannot find module 'xxx'`: パッケージのインストール忘れ。 `npm install` を再実行する - `EACCES: permission denied`: ファイルのアクセス権限の問題。再起動や `node_modules` の削除で直ることが多い

1.4 環境変数の確認

デプロイ前に、必要な環境変数がすべて揃っているか確認しましょう。

```
# .env.local の内容を確認する
# cat コマンド: ファイルの内容を画面に表示する
cat .env.local
```

必要な環境変数一覧:

変数名	説明	例
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase プロジェクトの URL	<code>https://abcdefg.supabase.co</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase の匿名キー	<code>eyJhbGciOiJIUzI1NiIs...</code>

重要: `.env.local` は `.gitignore` に含まれているため、GitHub にはプッシュされません。これはセキュリティ上正しい動作です。もし `.env.local` を間違えて GitHub にプッシュしてしまうと、世界中の人にキーが見られてしまうため、`.gitignore` で守られている、ということをお覚悟しておきましょう。デプロイ先（Vercel）で別途環境変数を設定する必要があります。

1.5 ビルド前チェックリスト

デプロイ前に以下をすべて確認しましょう。

- [] `npm run build` がエラーなく成功する
- [] `npm run lint` がエラーなく成功する
- [] `.env.local` に必要な環境変数がすべて設定されている
- [] `.gitignore` に `.env.local` と `node_modules` が含まれている

- [] ブラウザで全ページが正常に表示される
- [] CRUD 操作（作成・読取・更新・削除）がすべて動作する
- [] コンソールにエラーが出ていない

2. Vercel へのデプロイ

2.1 Vercel とは

Vercel（ヴァーセル）は、Next.js を開発している会社が提供するホスティングプラットフォームです。Next.js アプリケーションのデプロイに最適化されており、最も簡単かつ確実にデプロイできます。

Vercel を選ぶ理由:

- Next.js の開発元が運営しているため、最新機能への対応が最速
- GitHub と連携して、プッシュするだけで自動デプロイ
- 無料プラン（Hobby）で個人プロジェクトには十分な機能
- グローバル CDN で高速配信
- SSL（HTTPS）が自動で設定される
- プレビューデプロイ（Pull Request ごとにプレビュー URL を自動生成）

2.2 ホスティングサービスの比較

サービス	Next.js 対応	無料プラン	自動デプロイ	難易度	特徴
Vercel	最適	あり	あり	簡単	Next.js 公式。最も相性が良い
Netlify	良好	あり	あり	簡単	静的サイトに強い。Next.js 対応も改善中
AWS Amplify	良好	あり（制限付き）	あり	やや難	AWS エコシステムとの統合が強み
Railway	良好	あり（制限付き）	あり	普通	バックエンドも含めた総合プラットフォーム
Cloudflare Pages	部分的	あり	あり	やや難	エッジコンピューティングに強い

この教材では Vercel を使用します。Next.js との相性が最も良く、設定が最も簡単だからです。

2.3 事前準備: GitHub リポジトリの作成

Vercel にデプロイするには、コードが GitHub に存在する必要があります。まだ GitHub リポジトリを作成していない場合は、以下の手順で作成します。

Step 1: Git の初期化とコミット

```
# -----  
# プロジェクトのルートディレクトリ (package.json があるフォルダ) で実行する  
# -----  
  
# (1) Git リポジトリを初期化 (既に init 済みなら不要、何度実行してもOK)  
#   カレントフォルダに隠しフォルダ .git/ が作られ、Git管理が始まる  
#   git = バージョン管理ツールの名前  
#   init = initialize (初期化) の略  
git init  
# ▼ 出力例  
# Initialized empty Git repository in /path/to/book-management/.git/  
  
# (2) ステージングエリアに全ファイルを登録  
#   git add = コミット候補に追加するコマンド  
#   . はカレントフォルダ全体を意味する (「すべてのファイル」と読み替えてもOK)  
#   .gitignore に書かれたファイル/フォルダ (node_modules や .env.local 等) は  
#   自動的に除外されるので安心。  
git add .  
  
# (3) 最初のコミット  
#   commit = 「変更を記録する」操作。スナップショットを残すイメージ  
#   -m "... " はコミットメッセージ。何をした履歴か後で分かるよう短文で残す。  
#   メッセージなしで commit するとエディタが開く (初心者がハマりやすいポイント)  
git commit -m "書籍管理アプリの初回コミット"  
# ▼ 出力例  
# [main (root-commit) abc1234] 書籍管理アプリの初回コミット  
# 150 files changed, 12345 insertions(+)  
# create mode 100644 README.md  
# create mode 100644 package.json  
# ...
```

Step 2: GitHub にリポジトリを作成

1. [GitHub](#) にログインする
2. 右上の「+」ボタンをクリック → 「New repository」を選択
3. リポジトリ名を入力 (例: `book-management-app`)
4. 「Private」を選択 (公開したくない場合)
5. 「Create repository」をクリック

Step 3: ローカルリポジトリと GitHub を接続

```
# -----
# GitHub のリポジトリ作成画面に表示される3行のコマンド
# -----

# (1) リモート接続先を「origin」という名前で登録
#   git remote add = リモート (GitHub などのサーバー側リポジトリ) を登録する命令
#   origin = リモートの「あだ名」。慣習的に最初のリモートには origin を使う
#   URL は GitHub 上のリポジトリの URL に置き換える
#   これ以降「origin」と書けばそのURLを指すようになる
git remote add origin https://github.com/あなたのユーザー名/book-management-app.git

# (2) ローカルのブランチ名を「main」に統一する（古い環境では master の場合がある）
#   git branch = ブランチ操作のコマンド
#   -M は「強制リネーム」を意味する（大文字の M）
#   現在のブランチ名を main に変える
git branch -M main

# (3) 初回プッシュ
#   git push = ローカルの履歴をリモートに送る操作
#   -u は --set-upstream の短縮形。「これ以降のデフォルトの送信先」として origin の main を覚えさせる
#   origin = リモート名（さっき登録したGitHubのURLのあだ名）
#   main = 送るブランチ名
#   これを一度やれば、次回からは git push だけで済む
git push -u origin main

# ▼ 出力例
# Enumerating objects: 150, done.
# Counting objects: 100% (150/150), done.
# ...
# To github.com:yourname/book-management-app.git
# * [new branch]      main -> main
# branch 'main' set up to track 'origin/main'.
```

2.4 Vercel アカウントの作成

1. [Vercel の公式サイト](#) にアクセス
2. 「Sign Up」をクリック
3. 「Continue with GitHub」を選択して GitHub アカウントでログイン
4. Vercel が GitHub へのアクセス権限を要求するので「Authorize」をクリック
5. アカウント作成完了

注意: GitHub アカウントでログインすることで、リポジトリとの連携が簡単になります。

2.5 デプロイ手順 (Step by Step)

Step 1: 新規プロジェクトの作成

1. Vercel ダッシュボード (<https://vercel.com/dashboard>) にアクセス
2. 「Add New...」→「Project」をクリック
3. 「Import Git Repository」セクションで、先ほど作成した `book-management-app` リポジトリを見つける
4. 「Import」をクリック

Step 2: プロジェクトの設定

インポート画面で以下を確認します。

- Project Name: `book-management-app` (変更可能)
- Framework Preset: `Next.js` (自動検出されるはず)
- Root Directory: `./` (プロジェクトのルート)
- Build Command: `npm run build` (デフォルトのまま)
- Output Directory: `.next` (デフォルトのまま)

Build Command と Output Directory の意味: - **Build Command** (ビルドコマンド) : Vercel のサーバー上で実行されるコマンド。デフォルトの `npm run build` は package.json の `scripts.build` (中身は `next build`) を呼び出します - **Output Directory** (出力ディレクトリ) : ビルド結果が保存される場所。Next.js では `.next/` フォルダに最適化されたファイルが入る。Vercel はこのフォルダの中身をサーバーに配置して公開します

Step 3: 環境変数の設定

これが最も重要なステップです。環境変数を設定しないと、Supabase に接続できずアプリが動作しません。

1. 「Environment Variables」セクションを展開する
2. 以下の変数を追加する

Key	Value
<code>NEXT_PUBLIC_SUPABASE_URL</code>	<code>https://あなたのプロジェクトID.supabase.co</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	<code>eyJhbGciOiJIUzI1NiIs...</code> (あなたの匿名キー)

値の確認方法: Supabase ダッシュボード → Settings → API で確認できます。

3. 「Add」ボタンで各変数を追加

重要：環境変数は環境ごとに設定できる: Vercel では、環境変数を「Production（本番）」「Preview（プレビュー）」「Development（開発）」の3つの環境ごとに設定できます。多くの場合、すべての環境にチェックを入れて同じ値を入れておけばOKです。本番DBと開発DBを分けたい場合は、環境ごとに異なる値を設定することも可能です。

Step 4: デプロイの実行

1. 「Deploy」ボタンをクリック
2. Vercel がビルドとデプロイを開始する（通常 1〜3 分）
3. 成功すると、紙吹雪のアニメーションとともにデプロイ完了画面が表示される
4. 表示された URL（例: <https://book-management-app.vercel.app>）をクリックしてアプリを確認

（補足）Vercel CLI を使う場合

ブラウザではなくターミナルから直接デプロイしたい場合、Vercel CLI を使えます。GitHub 経由のデプロイで十分なので必須ではありませんが、参考までに紹介します。

```
# (1) Vercel CLI をグローバルインストール
# -g は global (システム全体に入れる) オプション
# これでターミナルのどこからでも vercel コマンドが使える
npm install -g vercel

# (2) Vercel アカウントでログイン
# ブラウザが開き、認証フローに進む
vercel login

# (3) 現在のディレクトリをプレビュー環境にデプロイ
# プロジェクトのルート (package.json があるフォルダ) で実行
vercel
# ↑ 初回は質問がいくつか出る (プロジェクト名・フレームワーク等)。基本Enterで進めればOK

# (4) 本番環境にデプロイ
# --prod = Production (本番) 環境にデプロイ
vercel --prod
```

（補足）vercel.json の例

プロジェクトのルートに `vercel.json` を置けば、Vercel の挙動を細かく設定できます。本書のチュートリアルでは不要ですが、参考までに。

```
{
  "framework": "nextjs",
  "buildCommand": "npm run build",
  "outputDirectory": ".next",
  "installCommand": "npm install",
  "devCommand": "npm run dev",
  "regions": ["hnd1"]
}
```

各行の意味：

- `"framework": "nextjs"` ... フレームワーク種別。Vercel が最適なビルド方式を自動選択するためのヒント
- `"buildCommand": "npm run build"` ... ビルド時に実行するコマンド
- `"outputDirectory": ".next"` ... ビルド結果の出力先フォルダ
- `"installCommand": "npm install"` ... 依存パッケージのインストール時に実行するコマンド
- `"devCommand": "npm run dev"` ... 開発サーバー起動時のコマンド（`vercel dev` で使われる）
- `"regions": ["hnd1"]` ... デプロイ先リージョン。`hnd1` は東京リージョン。日本のユーザーがメインなら東京を指定すると応答が速い

（補足）next.config.js の本番向け設定例

`next.config.js` は Next.js プロジェクトの設定ファイルです。本書ではほぼデフォルトでOKですが、本番向けに調整したい場合のサンプルです。

```
// JSDoc コメントで型情報を付ける (エディタの補完が効くようになる)
/** @type {import('next').NextConfig} */
const nextConfig = {
  // reactStrictMode = React の Strict Mode を有効にする
  // 開発中に副作用の問題を早期発見しやすくなる。本番では無効化されるので付けっぱなしでOK
  reactStrictMode: true,

  // poweredByHeader = レスポンスヘッダから X-Powered-By: Next.js を消す
  // 使用技術を外部に教えないことでセキュリティを少し高められる
  poweredByHeader: false,

  // compress = レスポンスをgzipで圧縮するか。デフォルトでtrue (基本そのまま)
  compress: true,

  // images = next/image コンポーネントの設定
  images: {
    // remotePatterns = 外部画像を許可するドメイン一覧
    // ここに登録されていないドメインの画像は <Image> で表示できない (セキュリティ対策)
    remotePatterns: [
      {
        // https のみ許可
        protocol: 'https',
        // Supabase の任意のサブドメインを許可
        hostname: '*.supabase.co',
        // /storage/ 以下のパスのみ許可
        pathname: '/storage/**',
      },
    ],
  },
};

// CommonJS の書き方で nextConfig を外部に公開する (Next.js が読み込む)
module.exports = nextConfig;
```

本番でアプリを直接起動するコマンド: Vercel では自動で行われますが、もし自前のサーバーで動かす場合は、ビルド後に `npm start` (中身は `next start`) を実行します。これは「ビルド済みの `.next/` フォルダの中身を使って本番モードで Web サーバーを起動する」コマンドです。 `npm run dev` と違い、ファイルを変更しても自動反映されません。

2.6 デプロイの自動化フロー

一度設定すれば、以降は GitHub にプッシュするだけで自動デプロイされます。



2.7 環境変数の追加・変更方法 (デプロイ後)

デプロイ後に環境変数を変更したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」タブ → 「Environment Variables」
3. 変数を追加・編集・削除
4. **重要:** 環境変数を変更した後は、「Deployments」タブから最新のデプロイを「Redeploy」する必要があります

再デプロイが必要な理由 (もう一度) : Next.js は `NEXT_PUBLIC_` 付きの環境変数を「ビルド時」にコード内に埋め込みます。値を変えただけではコード内の値は古いままなので、もう一度ビルドし直す必要があるのです。Redeploy ボタンを押すと、Vercel は最新コミットを使って再ビルドし、新しい環境変数の値で動くバージョンを公開します。

2.8 カスタムドメインの設定 (任意)

独自ドメイン (例: `mybooks.example.com`) を設定したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」タブ → 「Domains」
3. ドメイン名を入力して「Add」

4. 表示される DNS 設定をドメインレジストラ（お名前.com、Google Domains など）で設定
5. DNS の反映を待つ（通常数分～最大48時間）

初心者へ: カスタムドメインは必須ではありません。Vercel が自動生成する `xxx.vercel.app` の URL でも十分に利用できます。DNS（ドメインネームシステム）は、世界中のサーバーに新しい設定が伝わるまで時間がかかります。設定した直後に反映されなくても焦らず待ちましょう。

2.9 デプロイ後の動作確認

デプロイが完了したら、以下の項目を確認しましょう。

- ☐ トップページが正常に表示される
- ☐ 書籍一覧が Supabase から取得されて表示される
- ☐ 新しい書籍を追加できる
- ☐ 書籍の詳細ページが表示される
- ☐ 書籍の情報を編集できる
- ☐ 書籍を削除できる
- ☐ スマートフォンでも正常に表示される（レスポンス）
- ☐ HTTPS（鍵アイコン）で接続されている

トラブルシューティング: 問題が発生した場合は、Vercel ダッシュボードの「Deployments」→ 該当デプロイ → 「Logs」でビルドログとランタイムログを確認できます。ビルドログは「ビルド時のエラー」、ランタイムログは「ユーザーアクセス時のエラー」を見るのに使います。

3. Supabase の本番設定

アプリが本番環境で公開された後は、セキュリティとデータ保護をしっかりと確認する必要があります。

3.1 セキュリティの確認

API キーの種類を理解する

Supabase には2種類のキーがあります。

キーの種類	用途	公開してよいか
<code>anon</code> キー（匿名キー）	クライアントサイドからのアクセス	はい（RLS で保護）
<code>service_role</code> キー	サーバーサイドからの管理アクセス	絶対に公開してはいけない

NEXT_PUBLIC_SUPABASE_URL	→ クライアントに公開される (OK)
NEXT_PUBLIC_SUPABASE_ANON_KEY	→ クライアントに公開される (OK、RLS が必須)
SUPABASE_SERVICE_ROLE_KEY	→ サーバーサイドのみで使用 (絶対に公開しない)

重要: `NEXT_PUBLIC_` プレフィックスが付いた環境変数は、ブラウザの JavaScript から参照可能です。
`service_role` キーには絶対に `NEXT_PUBLIC_` を付けないでください。万一付けたままビルド・公開すると、悪意のあるユーザーがあなたのデータベースを全削除できてしまいます。

フロントエンドのコードで確認すべきこと

ブラウザの開発者ツール (F12 → Network タブ) で、以下を確認しましょう。

- Supabase への API リクエストが HTTPS で行われていること
- `service_role` キーがリクエストヘッダに含まれていないこと
- レスポンスに不要な個人情報が含まれていないこと

3.2 RLS (Row Level Security) ポリシーの見直し

RLS は Supabase のセキュリティの要です。本番環境では、適切なポリシーが設定されていることを必ず確認しましょう。

Auth Redirect URL の本番設定

将来的に認証機能を追加するときに必要な設定なので、ここで触れておきます。

1. Supabase ダッシュボード → Authentication → URL Configuration を開く
2. Site URL に本番 URL (例: `https://book-management-app.vercel.app`) を入れる
3. Redirect URLs にログイン後にリダイレクトされる URL を追加する (プレビュー環境用に `https://*.vercel.app` のようなワイルドカードも登録可能)

なぜ必要?: Supabase Auth は「ログイン後にどの URL に戻していいか」を厳格に管理しています。ここに登録されていない URL に戻そうとすると、エラーで弾かれます。ローカル (`http://localhost:3000`) と本番 (`https://...vercel.app`) の両方を登録しておくのが基本です。

現在の RLS 設定を確認する

Supabase ダッシュボード → Table Editor → `books` テーブル → 「RLS」タブで確認できます。

この教材では、認証なしで誰でもアクセスできる設定にしています。

```

-- 現在のポリシー（開発用・学習用）
-- -- で始まる行は SQL のコメント（実行されない）
-- すべてのユーザーが読み取り可能
-- ポリシー名を「誰でも書籍を読める」とし、books テーブルに適用
CREATE POLICY "誰でも書籍を読める" ON books
  -- FOR SELECT = 読み取り操作に対する許可。USING (true) = 常に許可
  FOR SELECT USING (true);

-- すべてのユーザーが書き込み可能
CREATE POLICY "誰でも書籍を追加できる" ON books
  -- FOR INSERT = 追加操作。WITH CHECK (true) = 常に許可
  FOR INSERT WITH CHECK (true);

-- すべてのユーザーが更新可能
CREATE POLICY "誰でも書籍を更新できる" ON books
  -- FOR UPDATE = 更新操作。常に許可
  FOR UPDATE USING (true);

-- すべてのユーザーが削除可能
CREATE POLICY "誰でも書籍を削除できる" ON books
  -- FOR DELETE = 削除操作。常に許可
  FOR DELETE USING (true);

```

本番環境向けの推奨設定

本格的なアプリケーションでは、認証を導入した上で以下のようなポリシーに変更することを推奨します。

```

-- 本番用ポリシー（認証導入後）
-- 誰でも読み取り可能（公開データの場合）
CREATE POLICY "誰でも書籍を読める" ON books
  -- 読み取りは全員OK
  FOR SELECT USING (true);

-- 認証済みユーザーのみ書き込み可能
CREATE POLICY "認証済みユーザーのみ追加可能" ON books
  -- auth.role() = 現在のユーザーの権限を返す関数。'authenticated' = ログイン済み
  FOR INSERT WITH CHECK (auth.role() = 'authenticated');

-- 自分が追加した書籍のみ更新可能
CREATE POLICY "自分の書籍のみ更新可能" ON books
  -- auth.uid() = ログイン中のユーザーID。user_id 列と一致する行だけ更新可
  FOR UPDATE USING (auth.uid() = user_id);

-- 自分が追加した書籍のみ削除可能
CREATE POLICY "自分の書籍のみ削除可能" ON books
  -- 自分のレコードしか消せない（他人のは触れない）
  FOR DELETE USING (auth.uid() = user_id);

```

この教材の範囲: 認証機能は次のステップとして紹介しますので、現時点では開発用のポリシーのままで問題ありません。ただし、**重要なデータを扱う本番アプリではセキュリティ設定を必ず強化してください。**

3.3 バックアップの設定

データベースのバックアップは、データ損失を防ぐために非常に重要です。

Supabase の自動バックアップ

- **Free プラン:** 自動バックアップなし（手動でのみ可能）
- **Pro プラン以上:** 毎日の自動バックアップ、Point-in-Time Recovery（PITR）

手動バックアップの方法

Supabase ダッシュボードから手動でバックアップを取得できます。

1. Supabase ダッシュボード → Settings → Database
2. 「Database Backups」セクション
3. 「Download backup」をクリック

または、`pg_dump` コマンドを使用する方法:

```
# Supabase のデータベース URL は、ダッシュボードの
# Settings → Database → Connection string で確認できます

# pg_dump = PostgreSQL に同梱されているバックアップ用コマンド (PostgreSQL クライアントのインストールが必要)
# "postgresql://..." = データベースへの接続文字列。"ユーザー名:パスワード@ホスト:ポート/データベース名" の形
# > backup.sql = 出力を backup.sql ファイルに書き込む (リダイレクト)。実行すると同じフォルダにバックアップが保存される
pg_dump "postgresql://postgres:[パスワード]@db.[プロジェクトID].supabase.co:5432/postgres"
> backup.sql
```

初心者へ: 学習目的であれば手動バックアップで十分です。本番アプリケーションを運用する際は、Pro プラン以上の利用を検討してください。

4. パフォーマンス最適化

デプロイしたアプリのパフォーマンスを向上させるためのテクニックを紹介します。

4.1 Next.js の Image コンポーネント

Next.js には、画像を自動的に最適化する `Image` コンポーネントが用意されています。書籍のカバー画像などを扱う場合に効果的です。

通常の `<img>` タグとの違い

```
// ❌ 通常の img タグ（最適化なし）
// 単純な HTML タグ。画像の自動最適化やレイアウトずれ防止は行われない


// ✅ Next.js の Image コンポーネント（自動最適化）
// next/image からインポートして使う。デフォルトエクスポートを Image という名前で受け取る
import Image from 'next/image';

<Image
  // 画像のパス（public/ フォルダ基準）
  src="/book-cover.jpg"
  // 代替テキスト（必須。アクセシビリティ・SEO 向上）
  alt="書籍カバー"
  // 元画像の幅（px単位）。レイアウトずれ防止に必須
  width={200}
  // 元画像の高さ（px単位）。レイアウトずれ防止に必須
  height={300}
  // 読み込み中にぼかし表示。"blur" か "empty" を指定
  placeholder="blur"
  // ぼかし画像のデータURL。低解像度のプレースホルダ用
  blurDataURL="data:..."
/>
```

Image コンポーネントのメリット:

機能	説明
自動リサイズ	デバイスに合わせた最適なサイズで配信
形式変換	WebP/AVIF など効率的な形式に自動変換
遅延読み込み	ビューポートに入るまで読み込みを遅延
CLS 防止	width/height の指定でレイアウトシフトを防止
キャッシュ	最適化した画像をキャッシュして再利用

外部画像を使用する場合

外部サイトの画像を使用する場合は、`next.config.js` に許可するドメインを追加する必要があります。

```
// next.config.js
// JSDoc コメント。TypeScript の型情報を JS ファイルに付与する記法
/** @type {import('next').NextConfig} */
const nextConfig = {
  // images = next/image の設定
  images: {
    // remotePatterns = 外部画像を許可するドメインパターンのリスト
    // ここに登録されていないドメインの画像は <Image> で読み込めない (セキュリティのため)
    remotePatterns: [
      {
        // https のみ許可 (http は不可)
        protocol: 'https',
        // 画像を取得するドメイン
        hostname: 'example.com',
        // /images/ 以下の任意のパスを許可 (** はワイルドカード)
        pathname: '/images/**',
      },
      {
        protocol: 'https',
        // Supabase の任意のサブドメインを許可。* は単一階層のワイルドカード
        hostname: '*.supabase.co',
        // /storage/ 以下を許可
        pathname: '/storage/**',
      },
    ],
  },
};

// CommonJS の書き方で外部に公開
// Next.js はこのファイルを require して、ここから設定オブジェクトを読み取る
module.exports = nextConfig;
```

4.2 メタデータの設定 (SEO)

検索エンジンにアプリを正しく認識してもらうために、メタデータを設定しましょう。

SEO とは？ Search Engine Optimization (検索エンジン最適化) の略。Google などの検索エンジンに、自分のサイトを正しく理解してもらい、検索結果に出やすくする工夫のこと。

ルートレイアウトでのメタデータ設定

```
// src/app/layout.tsx
// Metadata 型を next からインポート。TypeScript の型チェックに使う
import type { Metadata } from 'next';

// metadata という名前で export すると、Next.js が自動でメタタグを生成してくれる
// : Metadata は型注釈。間違ったキーを書くとエディタ・ビルド時にエラーになる
export const metadata: Metadata = {
  // title = <title> タグの内容を設定
  title: {
    // デフォルトのタイトル (titleが指定されていないページで使用)
    default: '書籍管理アプリ',
    // 各ページのタイトルが自動的に「ページ名 | 書籍管理アプリ」になる (%s が各ページのtitleで置き換わる)
    template: '%s | 書籍管理アプリ',
  },
  // <meta name="description"> 用
  description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  // <meta name="keywords"> 用 (現代では重要度は低い)
  keywords: ['書籍管理', '本', 'レビュー', 'Next.js'],
  // 著者情報
  authors: [{ name: 'あなたの名前' }],
  // openGraph = OGP (OpenGraph Protocol) の設定。SNS でシェアしたときの見た目に使われる
  openGraph: {
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
    // サイトの代表URL
    url: 'https://your-app.vercel.app',
    // サイト名
    siteName: '書籍管理アプリ',
    // 言語と地域
    locale: 'ja_JP',
    // コンテンツ種別。website / article など
    type: 'website',
  },
  // twitter = X (旧Twitter) でシェアされたときのカード設定
  twitter: {
    // 大きな画像付きカードを使用
    card: 'summary_large_image',
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  },
  // robots = 検索エンジンクローラへの指示
  robots: {
    // true = 検索結果に表示してOK
    index: true,
    // true = ページ内のリンクをたどってOK
    follow: true,
```

```
  },  
};
```

各ページでのメタデータ設定

```
// src/app/books/page.tsx  
import type { Metadata } from 'next';  
  
// このページ固有のメタデータを定義  
export const metadata: Metadata = {  
  // → 「書籍一覧 | 書籍管理アプリ」と表示される (layout.tsx の template が適用される)  
  title: '書籍一覧',  
  description: '登録されている書籍の一覧を表示します。',  
};
```

動的なメタデータ（書籍詳細ページ）

```
// src/app/books/[id]/page.tsx
import type { Metadata } from 'next';

// このページに渡される props の型を定義
type Props = {
  // URL の [id] 部分の値を文字列として受け取る
  params: { id: string };
};

// generateMetadata = 動的にメタデータを作る関数。Next.js が自動で呼び出す
// async = 中で await（非同期処理）を使えるようにする
// Promise<Metadata> = 「Metadataを返す非同期関数」という型注釈
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  // Supabase から書籍データを取得
  // .single() = 1件だけを取得するメソッド
  const { data: book } = await supabase
    // books テーブルから
    .from('books')
    // 全カラム取得
    .select('*')
    // id が params.id と一致する行
    .eq('id', params.id)
    // 1件だけ
    .single();

  // 取得した書籍情報からメタデータを作って返す
  return {
    // book が null の場合は「書籍詳細」を使う (?? = nullish coalescing 演算子)
    title: book?.title ?? '書籍詳細',
    // テンプレートリテラルで動的に作成
    description: `${book?.title} (${book?.author}) の詳細情報`,
  };
}
```

4.3 Lighthouse での計測方法

Google Lighthouse は、Web アプリのパフォーマンス、アクセシビリティ、SEOなどを総合的に評価するツールです。

Lighthouse の使い方

1. Chrome ブラウザでデプロイしたアプリを開く
2. F12 キーで開発者ツールを開く
3. 上部タブから「Lighthouse」を選択
4. 「Analyze page load」をクリック

5. 数十秒後に結果が表示される

評価項目

項目	説明	目標スコア
Performance	ページの読み込み速度	90 以上
Accessibility	アクセシビリティ（音声読み上げ対応など）	90 以上
Best Practices	Web 開発のベストプラクティス準拠	90 以上
SEO	検索エンジン最適化	90 以上

スコアを上げるコツ

- **Performance:** Image コンポーネントの使用、不要な JavaScript の削除、フォントの最適化
- **Accessibility:** alt 属性の設定、十分なカラーコントラスト、セマンティックな HTML
- **Best Practices:** HTTPS の使用、コンソールエラーの解消
- **SEO:** メタデータの設定、title タグの設定、レスポンシブデザイン

5. 学習の振り返り

ここまでお疲れさまでした。この教材を通じて、あなたは多くのことを学びました。一つずつ振り返ってみましょう。

5.1 この教材で学んだこと

以下のチェックリストで、あなたの学習成果を確認してください。

TypeScript の基礎

- [x] 変数の型注釈（ `string` , `number` , `boolean` ）
- [x] 型エイリアス（ `type Book = { ... }` ）
- [x] ジェネリクスの基本的使用
- [x] `interface` と `type` の違い
- [x] オプションプロパティ（ `?` ）
- [x] 型推論の仕組み

React の基礎

- [x] コンポーネントとは何か
- [x] JSX の書き方

- [x] Props の受け渡し
- [x] State の管理 (`useState`)
- [x] 副作用の処理 (`useEffect`)
- [x] イベントハンドリング (`onClick` , `onChange` , `onSubmit`)
- [x] 条件付きレンダリング
- [x] リストのレンダリング (`map` と `key`)
- [x] フォームの作成と制御

Next.js の基礎

- [x] App Router の仕組み (ファイルベースルーティング)
- [x] Server Components と Client Components の違い
- [x] `'use client'` ディレクティブの使い方
- [x] `page.tsx` と `layout.tsx` の役割
- [x] 動的ルート (`[id]`)
- [x] リンクとナビゲーション (`Link` コンポーネント)
- [x] メタデータの設定

Supabase でのデータベース操作

- [x] Supabase プロジェクトの作成
- [x] テーブルの作成
- [x] RLS (Row Level Security) の設定
- [x] データの取得 (`SELECT`)
- [x] データの追加 (`INSERT`)
- [x] データの更新 (`UPDATE`)
- [x] データの削除 (`DELETE`)
- [x] Supabase クライアントの初期化

CRUD アプリケーションの実装

- [x] Create: 新しい書籍の追加フォーム
- [x] Read: 書籍一覧の表示、詳細ページ
- [x] Update: 書籍情報の編集フォーム
- [x] Delete: 書籍の削除 (確認ダイアログ付き)
- [x] ローディング状態の処理
- [x] エラーハンドリング

Tailwind CSS でのスタイリング

- [x] ユーティリティクラスの基本
- [x] レスポンシブデザイン (`sm:` , `md:` , `lg:`)
- [x] Flexbox / Grid レイアウト
- [x] ホバー・フォーカスのスタイル
- [x] カラーパレットの使い方
- [x] カスタムコンポーネントのスタイリング

Vercel へのデプロイ

- [x] GitHub リポジトリの作成
- [x] Vercel アカウントの作成と連携
- [x] 環境変数の設定
- [x] 自動デプロイの仕組み
- [x] デプロイ後の動作確認

5.2 技術スタック全体像

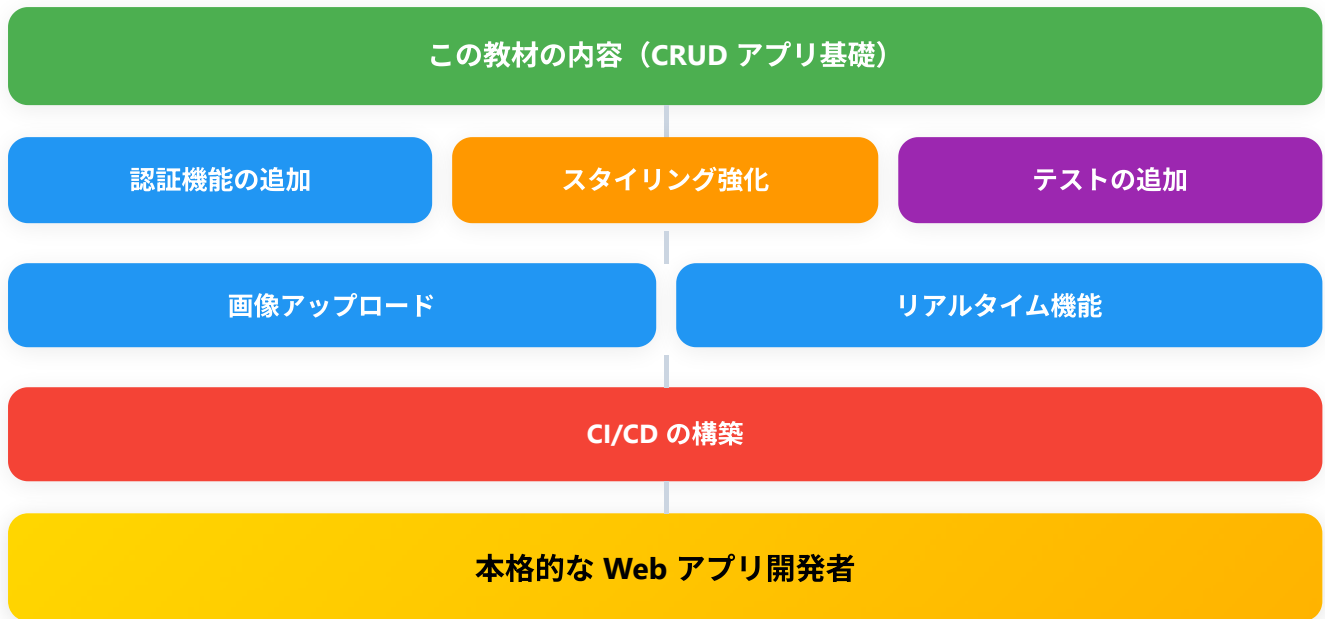
この教材で使用した技術スタックを図で確認しましょう。



6. 次のステップ（発展学習）

この教材で基礎を身につけたあなたは、さらに多くの機能を追加できます。以下に、次のステップとして取り組める発展的なトピックを紹介します。

6.1 学習ロードマップ



6.2 認証機能の追加 (Supabase Auth)

ユーザーがアカウントを作成し、ログインしてから書籍を管理できるようにします。

実装のイメージ:

```
// Supabase Auth を使ったログイン例
// supabase クライアントをインポート
import { supabase } from '@/lib/supabase';

// メールアドレス + パスワードでサインアップ（新規登録）
// supabase.auth.signUp = 新規ユーザー作成のメソッド。Promise を返すので await で待つ
// 戻り値の data には作成されたユーザー情報、error には失敗時の情報が入る
const { data, error } = await supabase.auth.signUp({
  // 登録するメールアドレス
  email: 'user@example.com',
  // パスワード（Supabase の最小文字数ポリシーを満たす必要あり）
  password: 'your-password',
});

// ログイン
// signInWithPassword = メール・パスワードでログインするメソッド
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'your-password',
});

// Google ログイン
// signInWithOAuth = 外部プロバイダ（Google, GitHub 等）でログインするメソッド
// 内部的にプロバイダの認証画面にリダイレクトされる
const { data, error } = await supabase.auth.signInWithOAuth({
  // 認証プロバイダの種類
  provider: 'google',
});

// ログアウト
// signOut = 現在のセッションを破棄するメソッド
const { error } = await supabase.auth.signOut();
```

学べること: - ユーザー認証の仕組み - セッション管理 - 保護されたルート（ログインしていないとアクセスできないページ） - RLS ポリシーとの連携

参考リソース: - [Supabase Auth ドキュメント](#) - [Next.js + Supabase Auth ガイド](#)

6.3 画像アップロード（Supabase Storage）

書籍のカバー画像をアップロードして表示する機能を追加します。

実装のイメージ:

```
// Supabase Storage を使った画像アップロード例
import { supabase } from '@lib/supabase';

// 画像のアップロード
// supabase.storage.from('バケット名') = 指定したバケット（保存場所）を操作する
// .upload(パス, ファイル, オプション) = ファイルをアップロードする
const { data, error } = await supabase.storage
  // バケット名（事前に Supabase ダッシュボードで作成しておく）
  .from('book-covers')
  // covers/ 以下に fileName でアップロード
  .upload(`covers/${fileName}`, file, {
    // ブラウザにキャッシュさせる秒数 (3600秒 = 1時間)
    cacheControl: '3600',
    // false = 同名ファイルがあればエラー。true なら上書き
    upsert: false,
  });

// アップロードした画像の公開 URL を取得
// getPublicUrl = バケットが Public 設定の場合に公開URLを取得する
const { data: { publicUrl } } = supabase.storage
  // バケット名
  .from('book-covers')
  // 取得したいファイルのパス
  .getPublicUrl(`covers/${fileName}`);
```

学べること: - ファイルアップロードの仕組み - Supabase Storage の使い方 - 画像のプレビュー表示 - ファイルサイズ・形式のバリデーション

参考リソース: - [Supabase Storage ドキュメント](#)

6.4 リアルタイム機能（Supabase Realtime）

複数ユーザーが同時にアプリを開いているとき、誰かが書籍を追加・編集すると、他のユーザーの画面にもリアルタイムで反映される機能です。

実装のイメージ:

```

// Supabase Realtime を使ったリアルタイム購読例
// useEffect を使うので Client Component にする必要がある
'use client';

import { useEffect } from 'react';
import { supabase } from '@lib/supabase';

// ページコンポーネント。default export で Next.js のルーティング対象にする
export default function BooksPage() {
  // useEffect = コンポーネントのマウント時に副作用を実行するフック
  // 第2引数 [] により、初回マウント時に1度だけ実行される
  useEffect(() => {
    // books テーブルの変更をリアルタイムで監視
    // .channel('チャンネル名') = WebSocket チャンネルを作成
    const channel = supabase
      // チャンネルの名前。任意の文字列
      .channel('books-changes')
      .on(
        // PostgreSQL の変更イベントを監視
        'postgres_changes',
        {
          // INSERT, UPDATE, DELETE すべて ('*' = 全種類)
          event: '*',
          // スキーマ名 (Supabase のデフォルトは 'public')
          schema: 'public',
          // 監視対象のテーブル名
          table: 'books',
        },
        (payload) => {
          // 変更が起きるたびに呼ばれるコールバック関数
          // payload には変更内容が入っている
          console.log('変更を検知:', payload);
          // ここで State を更新してUIに反映
        }
      )
    // 監視開始
    .subscribe();

    // クリーンアップ
    // useEffect が return する関数はコンポーネントのアンマウント時に呼ばれる
    return () => {
      // チャンネルを削除して購読停止 (メモリリーク防止)
      supabase.removeChannel(channel);
    };
  }, []);
}

```

```
return <div>...</div>;  
}
```

学べること: - WebSocket の仕組み - リアルタイムデータ同期 - 楽観的更新 (Optimistic Update)

参考リソース: - [Supabase Realtime ドキュメント](#)

6.5 テストの追加 (Jest, React Testing Library)

コードの品質を保証するために、テストを書く方法を学びます。

実装のイメージ:

```

// BookCard コンポーネントのテスト例
// render = コンポーネントをテスト用の仮想DOMに描画する関数
// screen = 描画された要素を取得するためのオブジェクト
import { render, screen } from '@testing-library/react';
import BookCard from '@components/BookCard';

// describe = 関連するテストをグループ化するブロック
// 第1引数: グループ名、第2引数: 中で it() を並べる関数
describe('BookCard', () => {
  // テスト用のダミーデータ
  const mockBook = {
    id: '1',
    title: '吾輩は猫である',
    author: '夏目漱石',
    rating: 5,
    created_at: '2025-01-01',
  };

  // it = 個別のテストケース。test() でも同じ
  // 第1引数: テストの説明、第2引数: 実行する関数
  it('書籍のタイトルが表示される', () => {
    // コンポーネントを描画
    render(<BookCard book={mockBook} />);
    // expect(...).toBeInTheDocument() = 要素が画面に存在することを確認
    // screen.getByText('文字列') = その文字列を含む要素を取得
    expect(screen.getByText('吾輩は猫である')).toBeInTheDocument();
  });

  it('著者名が表示される', () => {
    render(<BookCard book={mockBook} />);
    expect(screen.getByText('夏目漱石')).toBeInTheDocument();
  });

  it('評価が星で表示される', () => {
    render(<BookCard book={mockBook} />);
    // getAllByText = 該当する全要素を配列で取得
    const stars = screen.getAllByText('★');
    //toHaveLength(5) = 配列の長さが5であることを確認
    expect(stars).toHaveLength(5);
  });
});

```

学べること: - ユニットテスト・統合テストの考え方 - Jest テストランナーの使い方 - React Testing Library でのコンポーネントテスト - テスト駆動開発 (TDD) の基礎

参考リソース: - [Jest 公式ドキュメント](#) - [React Testing Library 公式ドキュメント](#)

6.6 CI/CD の構築

GitHub Actions を使って、コードをプッシュするたびに自動でテストを実行し、品質を保つ仕組みを構築します。

実装のイメージ:

```
# .github/workflows/ci.yml
# このファイルを置くだけで GitHub Actions が自動で読み込んでくれる
# YAML はインデント（半角スペース2つ）で階層を表す形式

# name = ワークフローの名前（GitHub の Actions タブに表示される）
name: CI

# on = いつこのワークフローを起動するか条件
on:
  push:
    # main ブランチに push されたとき
    branches: [main]
  pull_request:
    # main ブランチ向けの Pull Request が作られたとき
    branches: [main]

# jobs = 実行するジョブの定義（複数並列に書ける）
jobs:
  # ジョブの名前（任意）
  test:
    # 実行環境。最新の Ubuntu Linux を使う
    runs-on: ubuntu-latest
    # 順に実行するステップを並べる
    steps:
      # リポジトリのコードを取得する公式アクション
      - uses: actions/checkout@v4
      # Node.js をセットアップする公式アクション
      - uses: actions/setup-node@v4
        with:
          # Node.js のバージョン
          node-version: '20'
          # npm のキャッシュを使って高速化
          cache: 'npm'

      # npm ci = package-lock.json に従ってクリーンインストール
      - run: npm ci
      # lint チェックを実行
      - run: npm run lint
      # ビルドを実行（型エラーがないか確認）
      - run: npm run build
      # テストを実行
      - run: npm test
```

学べること: - CI/CD の概念と重要性 - GitHub Actions の使い方 - 自動テスト・自動デプロイのパイプライン - ブランチ戦略 (main, develop, feature ブランチ)

参考リソース: - [GitHub Actions 公式ドキュメント](#)

7. 推奨学習リソース

7.1 公式ドキュメント

リソース名	URL	説明
Next.js 公式ドキュメント	https://nextjs.org/docs	Next.js のすべての機能を網羅。チュートリアルも充実
React 公式ドキュメント	https://ja.react.dev	React の基礎から応用まで。日本語版あり
TypeScript 公式ドキュメント	https://www.typescriptlang.org/docs/	TypeScript の型システムを深く学べる
Supabase 公式ドキュメント	https://supabase.com/docs	Supabase の全機能のガイド
Tailwind CSS 公式ドキュメント	https://tailwindcss.com/docs	ユーティリティクラスの全リファレンス
Vercel 公式ドキュメント	https://vercel.com/docs	デプロイ・設定の詳細ガイド
MDN Web Docs	https://developer.mozilla.org/ja/	Web 技術全般のリファレンス。日本語版あり

7.2 おすすめの書籍

書籍名	対象	概要
『プロを目指す人のためのTypeScript入門』	初級～中級	TypeScript を体系的に学べる定番書
『React ハンズオンラーニング 第2版』	初級～中級	React の基礎を手を動かしながら学べる
『実践 Next.js』	中級	App Router を含む Next.js の実践的な解説
『Web API: The Good Parts』	中級	API 設計の基礎を学べる
『リアクト! TypeScript で始めるつらくない React 開発』	初級	React + TypeScript を優しく解説

7.3 コミュニティ

コミュニティ	URL	特徴
Zenn	https://zenn.dev	日本語の技術記事プラットフォーム。Next.js 関連の記事が豊富
Qiita	https://qiita.com	日本最大級の技術情報共有サービス
Stack Overflow	https://stackoverflow.com	世界最大の Q&A サイト。英語だが情報量が圧倒的
Discord（各技術の公式サーバー）	-	Next.js、Supabase 等それぞれ公式 Discord がある
X（旧 Twitter）	https://x.com	<code>#nextjs</code> <code>#supabase</code> などのハッシュタグで最新情報をキャッチ

8. 付録: 全ファイルの完成版一覧

この教材で作成したすべてのファイルの一覧です。

8.1 プロジェクト構成

```
book-management-app/  
├─ public/  
│   └─ favicon.ico  
├─ src/  
│   ├─ app/  
│   │   ├─ layout.tsx  
│   │   ├─ page.tsx  
│   │   ├─ globals.css  
│   │   └─ books/  
│   │       ├─ page.tsx  
│   │       ├─ new/  
│   │       │   └─ page.tsx  
│   │       └─ [id]/  
│   │           ├─ page.tsx  
│   │           └─ edit/  
│   │               └─ page.tsx  
│   ├─ components/  
│   │   ├─ BookCard.tsx  
│   │   ├─ BookForm.tsx  
│   │   ├─ Header.tsx  
│   │   ├─ DeleteButton.tsx  
│   │   └─ StarRating.tsx  
│   ├─ lib/  
│   │   └─ supabase.ts  
│   └─ types/  
│       └─ book.ts  
├─ .env.local  
├─ .gitignore  
├─ next.config.js  
├─ package.json  
├─ tailwind.config.ts  
├─ tsconfig.json  
└─ README.md
```

8.2 各ファイルの概要

ファイルパス	種類	説明
<code>src/app/layout.tsx</code>	レイアウト	アプリ全体の共通レイアウト。Header を含む
<code>src/app/page.tsx</code>	ページ	トップページ。アプリの紹介と書籍一覧へのリンク
<code>src/app/globals.css</code>	スタイル	Tailwind CSS のベーススタイル
<code>src/app/books/page.tsx</code>	ページ	書籍一覧ページ。Supabase からデータ取得して表示
<code>src/app/books/new/page.tsx</code>	ページ	新規書籍追加ページ。BookForm を使用
<code>src/app/books/[id]/page.tsx</code>	ページ	書籍詳細ページ。動的ルートでIDに基づく表示
<code>src/app/books/[id]/edit/page.tsx</code>	ページ	書籍編集ページ。既存データの更新
<code>src/components/BookCard.tsx</code>	コンポーネント	書籍カードの表示。一覧ページで使用
<code>src/components/BookForm.tsx</code>	コンポーネント	書籍の追加・編集フォーム。新規追加と編集で共用
<code>src/components/Header.tsx</code>	コンポーネント	ヘッダーナビゲーション
<code>src/components/DeleteButton.tsx</code>	コンポーネント	削除ボタン。確認ダイアログ付き
<code>src/components/StarRating.tsx</code>	コンポーネント	星評価の表示・入力コンポーネント
<code>src/lib/supabase.ts</code>	ユーティリティ	Supabase クライアントの初期化
<code>src/types/book.ts</code>	型定義	Book 型の定義
<code>.env.local</code>	環境変数	Supabase の接続情報（Git管理外）
<code>next.config.js</code>	設定	Next.js の設定ファイル
<code>tailwind.config.ts</code>	設定	Tailwind CSS のカスタム設定
<code>tsconfig.json</code>	設定	TypeScript のコンパイラ設定

9. おわりに

この教材を完走したあなたへ

この教材を最後まで読み進め、書籍管理アプリを完成させたあなたに、心からの祝福を送ります。

プログラミングの学習は、決して簡単な道のりではありません。新しい概念が次々と登場し、エラーに悩まされ、「自分には向いていないのかも」と思うこともあったかもしれません。それでも、ここまでたどり着いたということは、あなたには確実に「学び続ける力」があるということです。

振り返ってみてください。この教材を始めた時点のあなたと、今のあなたは全く違います。

- **TypeScript** で型安全なコードが書けるようになりました
- **React** でインタラクティブな UI を構築できるようになりました
- **Next.js** でモダンな Web アプリケーションを作れるようになりました
- **Supabase** でデータベースを操作できるようになりました
- **Tailwind CSS** で美しいデザインを実装できるようになりました
- **Vercel** にデプロイして、世界中からアクセスできるアプリを公開できるようになりました

これらはすべて、現在のWeb開発の現場で実際に使われている技術です。あなたは、実務で通用するスキルの基礎をすでに手に入れています。

大切なこと

プログラミング学習で最も大切なのは、**手を動かし続ける**ことです。

- この教材のアプリに新しい機能を追加してみてください
- 別のアプリのアイデアを考えて、ゼロから作ってみてください
- エラーに出会ったら、それは学びのチャンスです
- わからないことがあれば、公式ドキュメントを読む習慣をつけてください
- 完璧を求めず、まず動くものを作り、少しずつ改善していくことが大切です

「できない」は「まだできない」

今の時点で理解しきれていないことがあっても、心配はいりません。「できない」のではなく、「まだできない」だけです。プロのエンジニアも、毎日新しいことを学び続けています。学習に終わりはありませんが、それこそがプログラミングの面白さでもあります。

あなたが作った書籍管理アプリは、あなたのポートフォリオの第一歩です。これからもっと多くのアプリを作り、スキルを磨いていってください。

フィードバック

この教材に関するご意見・ご感想・誤りの報告は、以下の方法でお寄せください。

- **GitHub Issues**: このリポジトリの Issues ページで報告
- **Pull Request**: 修正提案がある場合は PR を作成

皆さまのフィードバックが、この教材をより良いものにします。

お疲れさまでした。あなたの Web 開発の旅は、ここから始まります。